

# Linear and logistic regression

## Lab 01: from response times to alert decisions

### Purpose and learning outcomes

Build two small models for a request-processing service. First predict a numerical response time. Then estimate the probability of a latency-budget violation and decide which requests should trigger an alert. The emphasis is on interpreting predictions, using unseen data honestly, and connecting mistakes to their consequences.

After completing the lab you should be able to:

- fit and interpret a simple linear model and compare it with a mean-only baseline;
- measure error in meaningful units and inspect residual patterns;
- fit a scaled logistic model without leaking validation or test information;
- distinguish a probability estimate from a threshold-based decision;
- explain why the most accurate decision policy need not have the lowest cost.

### Files and preparation

Open `lab_linear_logistic_student.ipynb`. It contains the data generators, imports, supplied checks, and nine incomplete code cells labeled A1–A4 and B1–B5. Replace each `NotImplementedError` block with your code. Later cells depend on those results, so an unfinished notebook is expected to stop at its first `TODO`. Complete the six written-answer prompts as well.

The separate `lab_linear_logistic_solution.ipynb` is a fully executed reference for review after attempting the exercises. Both examples use fixed, locally generated synthetic data. No external dataset or GPU is required. Use the same Python environment as the course notebook; the course's `requirements.txt` lists dependencies.

| Part                   | Time   | Main challenge                                           |
|------------------------|--------|----------------------------------------------------------|
| A: linear regression   | 35 min | Fit a line, measure improvement, explain errors.         |
| B: logistic regression | 75 min | Separate model selection, policy selection, and testing. |
| Review                 | 10 min | Run from a fresh kernel and explain the evidence.        |

### Small formula reference

A linear model predicts  $\hat{y} = b + wx$ . A residual is  $y - \hat{y}$ .

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}.$$

Logistic regression turns a weighted score into a probability:

$$z = b + \sum_j w_j x_j, \quad p = \frac{1}{1 + e^{-z}}, \quad \hat{y} = \begin{cases} 1, & p \geq t, \\ 0, & p < t. \end{cases}$$

The threshold  $t$  is a decision choice. Changing it does not require changing  $p$ .

## Example A: predict request latency

You have 120 independent simulated requests. The input `size_mb` is known before processing; the target `latency_ms` is observed afterward. Use a line to describe the average relationship. Individual requests still vary, so a good line need not pass through every point.

### A1: Select data, split, and inspect

Create a one-column DataFrame `Xa` containing `size_mb` and a target Series `ya` containing `latency_ms`. Use `train_test_split` with `test_size=0.25` and `random_state=SEED`. Name the results `Xa_train`, `Xa_test`, `ya_train`, and `ya_test`. Keep their original row indices. There should be 90 training and 30 test requests.

Plot training size against training latency. Label axes with units. Before fitting, state whether a straight-line trend seems plausible. Keep the target out of the input features, and do not show test data to decide on a different model family.

### A2: Fit a line and a meaningful baseline

Fit a `LinearRegression` model named `linear_model`. Also fit a `DummyRegressor(strategy="mean")` named `mean_model`. Both learn from training data only. Create `pred_a` and `baseline_a`, their predictions for the test inputs. Display the learned intercept and slope.

The baseline must predict the *training* mean. Computing the test mean and using it as the baseline would give that predictor information unavailable before evaluation.

### A3: Compare errors and visualize predictions

Create `regression_results`, indexed by `Baseline` and `Linear regression`, with columns `MAE_ms` and `RMSE_ms`. Evaluate both on the same test rows. Plot the training points, test points, fitted line, and mean-only baseline over the observed size range. Include a legend.

**Explain.** Interpret the intercept and slope with units. Does request size improve held-out performance compared with the baseline? Why may the points miss the line?

Do not describe RMSE as a bound that contains every request's error. It summarizes squared errors, and large mistakes have greater influence than with MAE.

### A4: Inspect errors and predict one new request

Create `residual_a = ya_test - pred_a`. Plot it against test request size with a horizontal zero line. Create `latency_4mb`, the prediction for a 4 MB request. Pass a DataFrame with the same feature name used in training.

**Explain.** What does a positive residual mean? Can a small residual plot prove that all assumptions are correct? Would you trust the prediction for a 1,000 MB request?

This final residual plot is an evaluation diagnostic. If it motivates redesign, you would need a new final evaluation rather than repeatedly tuning to these test errors.

**Expected outputs for A:** three labeled figures, a two-model error table, the fitted coefficients, one new prediction, and two short written responses.

## Example B: choose which requests trigger an alert

A second dataset contains 1,200 independently simulated requests. Predict `budget_violation`, where 1 means a latency budget is exceeded. All four input features are measured when the request arrives:

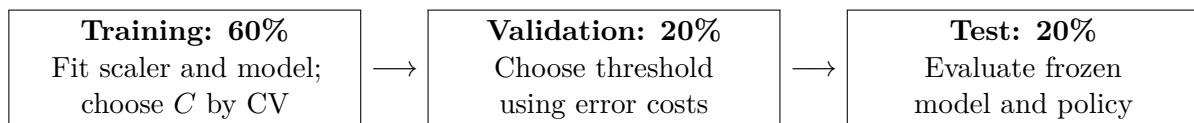
| Feature                   | Meaning                                                      |
|---------------------------|--------------------------------------------------------------|
| <code>payload_mb</code>   | Payload size in megabytes.                                   |
| <code>queue_length</code> | Number of requests already waiting.                          |
| <code>cpu_percent</code>  | CPU utilization at arrival, from 0 to 100.                   |
| <code>cache_hit</code>    | A lookup has already found a reusable cached result: 0 or 1. |

The simulator is supplied for reproducibility. Its internal generating score is not a permitted input feature or a substitute for learned predictions. The outcome is stochastic, so knowing the recorded inputs does not make every label predictable.

**Error costs.** A false alert (FP) costs 1 unit. A missed violation (FN) costs 5 units. Correct decisions cost zero. Your policy-selection objective is

$$\text{cost} = \text{FP} + 5 \text{FN}.$$

A positive prediction means “issue an alert.” Most observations are negative, so an always-no-alert policy can have respectable accuracy while missing every violation.



### B1: Create three disjoint partitions

Select the supplied `FEATURES` into `Xb`, and the target into `yb`. First hold out 20% as test data, leaving development data. Next hold out 25% of *development* data as validation. Use `random_state=SEED` and stratify on the corresponding target for both splits. Preserve row indices. The notebook specifies the required variable names. Sizes should be 720, 240, and 240.

Inspect training class counts and feature means/standard deviations.

**Explain.** Why might an always-no-alert policy be accurate yet costly? Why is a random split appropriate for this simulator but not necessarily for real service logs?

### B2: Select a scaled logistic model using training data only

Build a pipeline of `StandardScaler()` and `LogisticRegression(max_iter=2000)`. Search  $C \in \{0.01, 0.1, 1, 10\}$  using five-fold stratified cross-validation, shuffling with `random_state=SEED`. Use `scoring="neg_log_loss"` and fit on training rows only. Store the best fitted pipeline as `selected_model`; display selected  $C$  and CV log losses.

**Explain.** Why put the scaler inside the search pipeline? Which direction of  $C$  means stronger regularization? Why evaluate probabilities with log loss before selecting a threshold?

## Example B continued: probabilities and policy selection

### B3: Read probabilities before making decisions

Create `val_probability` using `selected_model.predict_proba(Xb_val)[: , 1]`. This selects the probability of class 1. Form `val_labels_05` with threshold 0.5 and display `validation_summary_05`: accuracy, precision, recall, and log loss. Use `zero_division=0` for precision and recall when needed.

Plot histograms of validation probabilities for actual negative and positive requests, using common bins and a legend. Mark 0.5 with a vertical line. Overlap means no threshold can perfectly separate these particular observations. It is not automatically a coding error.

| Quantity  | Question it answers                                                               |
|-----------|-----------------------------------------------------------------------------------|
| Accuracy  | What fraction of all decisions are correct?                                       |
| Precision | Of the requests that triggered alerts, how many actually violated the budget?     |
| Recall    | Of the actual violations, how many triggered alerts?                              |
| Log loss  | How much probability did the model assign to the outcomes that actually happened? |

For an observed positive case, log loss is  $-\log p$ ; for an observed negative case, it is  $-\log(1 - p)$ . A confident mistake is penalized more than an uncertain one. Log loss uses probabilities, not thresholded labels.

### B4: Choose the threshold by validation cost

Use the supplied thresholds 0.05, 0.10, ..., 0.95, without changing the grid. At each threshold, predict class 1 when  $p \geq t$  and compute FP, FN, and  $FP + 5FN$ . Store the results in `threshold_results` with columns `threshold`, `FP`, `FN`, and `cost`.

Use `confusion_matrix(..., labels=[0, 1]).ravel()` to obtain `tn`, `fp`, `fn`, `tp`. Select `best_threshold` with the lowest validation cost. **Tie rule: if several thresholds have equal minimum cost, choose the largest.** Plot cost against threshold, mark the selected threshold, and display the FP/FN counts for it and for the 0.5 reference.

**Explain.** Which mistakes does the selected policy trade for others? Does a lower validation cost guarantee the same improvement on new data? Does changing the threshold change the fitted probabilities?

#### Checkpoint before opening the test results.

- Model hyperparameters were chosen using training-only CV.
- The threshold was chosen using validation labels and the stated costs.
- No test labels were used for preprocessing, fitting, or selection.
- Model and threshold are now fixed. Do not refit on train plus validation: a refit could change the probability scale, so the selected threshold may no longer behave the same way.

Lowering a threshold expands the positive set. Recall cannot decrease for fixed probabilities, but precision need not move monotonically. Choose according to the specified costs rather than assuming that any one metric is always the goal.

## Final evaluation and submission

### B5: Evaluate the frozen policies on the test set

Create `test_probability` once from the selected model. Compare exactly these three predeclared policies on the same test rows:

1. `Always no alert`: predict class 0 for every request.
2. `Threshold 0.5`: threshold the fitted probabilities at 0.5.
3. `Validation threshold`: threshold those same probabilities at `best_threshold`.

Create `test_results` indexed by those names. Include `accuracy`, `precision`, `recall`, `FP`, `FN`, `cost`, and `cost_per_request`. Divide total cost by the number of test requests for the last column. Report the logistic model's log loss separately; both logistic policies share it because their probabilities are identical.

Plot the two logistic confusion matrices side by side. Label the true and predicted classes clearly. The always-no-alert baseline is a hard decision policy for the cost comparison; no probability-quality claim is required for that baseline.

**Explain.** Recommend a policy under the stated costs, using the validation choice and the held-out evidence. Is the highest-accuracy policy also the lowest-cost one? Why is log loss unchanged by the threshold? Name one limitation of the experiment.

Report what happened even if a validation-selected policy does not win on test data. Do not retune after observing these test results.

## What to submit

Submit the completed, executed student notebook with all nine code tasks and six written answers. Keep the supplied seeds, split proportions, feature list, threshold grid, and costs. Restart the kernel and run all cells in order; save the outputs. Include all six required figures and the comparison tables. Explain A's errors in milliseconds and B's errors in operational consequences.

| Assessment          | Points | Evidence                                                                    |
|---------------------|--------|-----------------------------------------------------------------------------|
| A1–A4               | 30     | Correct fit, baseline, metrics, units, plots, and new prediction.           |
| B1–B2               | 22     | Disjoint stratified partitions; training-only pipeline and model selection. |
| B3–B4               | 20     | Correct probability evaluation and validation-cost threshold selection.     |
| B5                  | 8      | Frozen test comparison with consistent cost calculations.                   |
| Six written answers | 20     | Clear interpretation, tradeoffs, and limits of the evidence.                |

Assess reasoning and a sound protocol, not an artificially high test score or identical last decimals. Supplied checks help catch common mistakes but do not replace explanation.

**Optional extension.** With calibrated probabilities, compare the expected costs  $c_{FP}(1 - p)$  and  $c_{FN}p$  to derive a threshold. Explain why a finite validation-grid optimum can differ. Keep any exploration on validation data rather than repeatedly reopening the test set.

**Sources.** Both simulators and examples were written for this lab. See the companion course notebook for concepts and the official [linear-model](#), [pipeline](#), and [evaluation](#) guides for API background.